Tag-System Scheduled Transformer Inference: A 2-Tag System as the Control Model for LLM Generation

ALVARO VIDELA, Microsoft

We present a transformer inference engine whose control flow is governed entirely by a 2-tag system in the sense of Post (1943). The engine maintains a two-symbol word $[OP, STATE]$ and advances by reading the operation symbol, executing the corresponding production rule (one neural-network sub-operation), and emitting the next $[OP', STATE']$ pair. For a 24-layer Qwen 2.5 0.5B model, each generated token requires exactly 315 tag steps.

The production rules encode sign-plus-magnitude matrix–vector products in the log domain, where weights are stored as $(\ln|w|, \mathrm{sign}(w))$ pairs and accumulation uses the log-sum-exp identity. We apply five optimizations drawn from the classical computer science literature: (a) a peephole-rewritten fast exp approximation (Aho et al., Dragon Book), (b) semigroup tree reduction for instruction-level parallelism (Stepanov & McJones), (c) constraint-propagation pruning at threshold e^{-20} (Dechter), (d) Grand Central Dispatch parallel column decomposition, and (e) a Metal GPU backend with a SIMD-cooperative kernel using half-precision magnitudes and packed sign bits.

We further introduce the `.tag` model format: a GPU-native, `mmap`-friendly binary encoding with page-aligned f16 magnitudes and packed sign bits, enabling zero-copy Metal buffer registration via `newBufferWithBytesNoCopy`. The format reduces model size from 4.8 GB (f64 `.eml`) to 2.0 GB (58% reduction) and eliminates the GPU upload phase entirely.

On an Apple M4 Max, the engine generates correct English text at ~12 tok/s wall-clock, with the matmul kernel achieving a 97.8× speedup over the baseline CPU implementation. The entire system—tag-system scheduler, five optimization layers, GPU bridge, and `mmap` loader—is implemented in approximately 1700 lines of C and 350 lines of Objective-C.

Additional Key Words and Phrases: tag system, Post production system, transformer inference, sign-magnitude arithmetic, Metal GPU, mmap, zero-copy

1 Introduction

Post's tag systems [1] are among the simplest known Turing-complete computational models. A m-tag system operates on a finite alphabet A with a production function $P : A \to A^*$: at each step, the machine reads the leftmost symbol a of the word, appends $P(a)$ to the right end, and deletes the leftmost m symbols. Cocke and Minsky [2] proved that 2-tag systems ($m = 2$) are universal.

Modern large language models (LLMs) are typically implemented as imperative programs with nested loops over layers, attention heads, and sequence positions. The control flow is implicit in the loop structure. We ask: *can a tag system serve as the explicit scheduling mechanism for transformer inference?*

We answer affirmatively by constructing EMILIO-TAG, an inference engine for Qwen 2.5 0.5B Instruct [3] in which:

(1) The computation state is a two-symbol word $[OP, STATE]$, where OP determines which neural-network sub-operation to execute and $STATE$ carries the mutable activation vectors.
(2) Each of 16 production rules performs one transformer sub-operation (embedding lookup, RMSNorm, QKV projection, RoPE, attention, FFN, etc.) and emits the next operation symbol.
(3) The word length is invariant at 2. The tag system halts after exactly $1 + 24 \times 13 + 2 = 315$ steps per token.
(4) The production rules are *parameterised* by model weights: $P(\text{QKV_MATMUL}, \text{layer} = k)$ uses the QKV weight matrix of layer k.

Author's Contact Information: Alvaro Videla, Microsoft.

The tag system governs sequential control flow—which operation runs after which—while the evaluation of each production rule uses conventional CPU or GPU arithmetic. We call this architecture *tag-system-scheduled, natively-evaluated.*

The rest of this paper describes the formal model (Section 2), the sign-magnitude matmul kernel and its five optimization layers (Section 3), the Metal GPU backend (Section 4), the `.tag` model format with `mmap` and zero-copy GPU registration (Section 5), experimental results (Section 6), and related work (Section 8).

2 The 2-Tag System Model

2.1 Formal Definition

A 2-tag system is a triple (m, A, P) where:

- $m = 2$ is the *deletion number.*
- A is a finite *alphabet.*
- $P : A \to A^*$ maps each symbol to a *production* (a finite string over A).

The machine maintains a *word* $w \in A^*$. At each step:

(1) Read the leftmost symbol $a = w_0$.
(2) If a is a halting symbol, stop.
(3) Append $P(a)$ to the right end of w.
(4) Delete the leftmost $m = 2$ symbols.

2.2 Instantiation for Transformer Inference

We define the alphabet for a transformer with L layers:

$$\begin{aligned} A = \{ & \textsc{embed}, \ \textsc{rmsnorm_attn}, \ \textsc{qkv_matmul}, \ \textsc{bias_add}, \\ & \textsc{rope}, \ \textsc{kv_store}, \ \textsc{attention}, \ \textsc{out_proj}, \\ & \textsc{residual_attn}, \ \textsc{ffn_norm}, \ \textsc{gate_up}, \ \textsc{silu_mul}, \\ & \textsc{down_proj}, \ \textsc{residual_ffn}, \ \textsc{final_norm}, \ \textsc{lm_head}, \\ & \textsc{halt}, \ \textsc{state} \} \end{aligned}$$

Each symbol carries a layer index $\ell \in \{0, \ldots, L-1\}$. The word is always exactly two symbols:

$$w = [OP_\ell, \ \textsc{state}]$$

The STATE symbol carries a mutable payload: the current activation vector $\mathbf{x}$, a residual vector, the sequence position, and the current layer index.

2.3 Production Rules

Each production $P(OP_\ell)$ performs one sub-operation and determines the next operation symbol. The key rules are:

Embedding. $P(\textsc{embed})$: look up $\mathbf{x} \leftarrow E[\text{token_id}]$, emit $[\textsc{rmsnorm_attn}_0, \textsc{state}]$.

Layer transition. $P(\textsc{residual_ffn}_\ell)$: compute $\mathbf{x} \leftarrow \mathbf{x}_{\text{residual}} + \mathbf{x}_{\text{ffn}}$. If $\ell < L-1$, emit $[\textsc{rmsnorm_attn}_{\ell+1}, \textsc{state}]$ otherwise emit $[\textsc{final_norm}, \textsc{state}]$.

Halting. $P(\textsc{lm_head})$: compute output logits via matmul, set argmax, emit $[\textsc{halt}, \textsc{state}]$.

2.4 Step Count Analysis

For one token through a 24-layer model:

$$\begin{aligned}\text{steps} &= 1_{\text{embed}} + L \times 13_{\text{layer ops}} + 1_{\text{final_norm}} + 1_{\text{lm_head}} \\ &= 1 + 24 \times 13 + 2 = 315\end{aligned}$$

The 13 layer operations are: RMSNORM_ATTN, QKV_MATMUL, BIAS_ADD, ROPE, KV_STORE, ATTENTION, OUT_PROJ, RESIDUAL_ATTN, FFN_NORM, GATE_UP, SILU_MUL, DOWN_PROJ, RESIDUAL_FFN.

3 Sign-Magnitude Arithmetic and Optimizations

3.1 Log-Domain Sign-Magnitude Representation

Weights are stored as pairs $(\ln|w_k|, \text{sign}(w_k))$. A matrix–vector product $\mathbf{y} = W\mathbf{x}$ becomes:

$$y_j = \sum_k \text{sign}(w_{jk}) \cdot \text{sign}(x_k) \cdot \exp\big(\ln|w_{jk}| + \ln|x_k|\big) \tag{1}$$

At runtime, the activation $\mathbf{x}$ is converted to $(\ln|x_k|, \text{sign}(x_k))$ before each matmul. The inner products are computed via log-sum-exp with a running maximum for numerical stability.

3.2 Optimization 1: Fast Exponential (Dragon Book)

The inner loop calls exp() approximately K times per dot product, where K is the inner dimension (896 for most layers, 4864 for DOWN_PROJ). We replace the libm `exp()` with a range-reduced 5th-degree polynomial [5]:

(1) **Range reduction**: $x = n \cdot \ln 2 + r$, where $n = \lfloor x/\ln 2 \rfloor$ and $0 \le r < \ln 2$.
(2) **Polynomial**: $\exp(r) \approx 1 + r(1 + r(0.5 + r(\frac{1}{6} + r(\frac{1}{24} + \frac{r}{120}))))$.
(3) **Reconstruction**: 2^n via IEEE 754 exponent bit manipulation (zero floating-point operations).

The relative error is below 2×10^{-7}, well within the tolerance of the sign-magnitude representation. This is a *peephole optimization* in the Dragon Book sense: a local pattern (expensive `exp` call) is replaced by a cheaper equivalent without changing the surrounding code.

3.3 Optimization 2: Pruning Threshold (Dechter)

Terms where $\ln|w_{jk}| + \ln|x_k| - \max < -20$ contribute at most $e^{-20} \approx 2 \times 10^{-9}$ to the sum. Over $K = 896$ terms, the maximum accumulated error is $K \cdot e^{-20} \approx 1.8 \times 10^{-6}$. We skip these terms entirely.

This is constraint propagation in the sense of Dechter [7]: propagating the constraint "this term's contribution is negligible" to prune the computation. The threshold was tightened from -30 to -20 after analyzing the error budget.

3.4 Optimization 3: Semigroup Tree Reduction (Stepanov)

The accumulation `acc = acc + f(x_k)` is a linear dependency chain of depth K. Since addition is associative (it forms a semigroup [6]), we can reparenthesise:

Listing 1. 4-way ILP accumulation

```
double a0 = 0, a1 = 0, a2 = 0, a3 = 0;
for (k = 0; k < K; k += 4) {
    a0 += fast_exp(s[k])   * sign[k];
    a1 += fast_exp(s[k+1]) * sign[k+1];
    a2 += fast_exp(s[k+2]) * sign[k+2];
    a3 += fast_exp(s[k+3]) * sign[k+3];
```

```
}
acc = (a0 + a1) + (a2 + a3);  // tree reduce
```

The final tree-reduce `(a0+a1)+(a2+a3)` has depth 2 instead of K. The same 4-way decomposition is applied to the max-finding pass.

3.5 Optimization 4: GCD Parallel Dispatch

The column loop of the matrix–vector product is embarrassingly parallel: each output element y_j depends on a different column of W. We split the columns across N worker threads using macOS Grand Central Dispatch (`dispatch_apply_f`):

Listing 2. GCD parallel dispatch

```
dispatch_apply_f(nchunks,
    dispatch_get_global_queue(
        QOS_CLASS_USER_INTERACTIVE, 0),
    &ctx, matmul_dispatch_work);
```

Thread count is auto-detected via `sysconf(_SC_NPROCESSORS_ONLN)` and overridable with `EML_THREADS`. The GCD thread pool has zero creation/destruction overhead across the ~3000 matmul calls per 8-token generation.

4 Metal GPU Backend

4.1 Architecture

The tag system determines *which* operation to execute; the GPU determines *how* the heaviest operations (matmuls) are evaluated. When a production rule calls `build_matmul_sm` and a GPU weight buffer exists (`gpu_weight_id` ≥ 0), the call is dispatched to the Metal compute shader. All other production rules (RMSNorm, softmax, RoPE, residual adds) run on the CPU.

This separation is principled: the tag system is the *control model* and the GPU is the *evaluation model*. The tag system still executes all 315 sequential steps per token—parallelism exists only *within* a single production rule's evaluation.

4.2 SIMD-Cooperative Kernel

The Metal shader assigns one SIMD group (32 threads) per output element (i, j). Each thread processes $\lceil K/32 \rceil$ elements of the inner dimension and contributes to a `simd_sum` reduction:

Listing 3. Metal kernel core (simplified)

```
// 4-way unrolled inner loop
for (; k + 96 < inner; k += 128) {
    float m0 = ln_a_mag[ak]   + half(ln_b_mag[bk]);
    float m1 = ln_a_mag[ak+32]+ half(ln_b_mag[bk+32]);
    float m2 = ln_a_mag[ak+64]+ half(ln_b_mag[bk+64]);
    float m3 = ln_a_mag[ak+96]+ half(ln_b_mag[bk+96]);
    // sign extraction from packed u32
    float sb0 = 1-2*float((w0>>lane)&1);
    acc0 += exp(m0) * (sign_a[ak] * sb0);
    ...
}
float total = simd_sum(acc0+acc1+acc2+acc3);
```

4.3 Weight Storage on GPU

Weight magnitudes are stored as f16 (half-precision), and signs are packed as single bits into u32 words (bit k of word $\lfloor k/32 \rfloor$ is 1 iff $w_k < 0$). This achieves a 23% bandwidth reduction over storing magnitudes as f32 and signs as f32.

The activation vector $\mathbf{x}$ is converted to $(\ln |x_k|, \mathrm{sign}(x_k))$ on the CPU and written to shared Metal buffers. The result is read back as f32 and converted to f64 for the next tag step.

5 The `.tag` Model Format

5.1 Motivation

The original `.eml` format stores weights as f64 sign-magnitude pairs. Loading the 0.5B model requires:

(1) Reading 4.8 GB of f64 data from disk (~10 s).
(2) Converting each weight from f64 to f16 magnitude and packing signs into u32 words.
(3) Copying the converted data to Metal GPU buffers (~6 s).

We designed the `.tag` format to eliminate steps 2 and 3.

5.2 Format Layout

The `.tag` file has four sections:

Header (84 bytes): Magic `TAG1`, version (u32), model configuration (10×u32 + 2×f64), tensor count (u32), and offsets to the tokenizer, tensor directory, and data sections.

Tokenizer: Vocabulary strings and BPE merge table, stored as length-prefixed byte sequences.

Dense arrays: Token embeddings, layer norms, and biases as f64 arrays with u64 length prefixes.

Tensor data (page-aligned): f16 magnitudes and packed u32 sign bits for all 97 weight matrices, laid out contiguously starting at a 16 KB-aligned offset.

A *tensor directory* between the dense arrays and tensor data provides the inner dimension, column count, magnitude offset, and sign offset for each tensor.

5.3 Zero-Copy GPU Registration

The tensor data section is page-aligned (16 KB on Apple Silicon) so that the `mmap`'d region can be wrapped directly as a Metal buffer:

Listing 4. Zero-copy Metal buffer from mmap

```
id<MTLBuffer> buf = [device
    newBufferWithBytesNoCopy:(void *)ptr
        length:nbytes
        options:MTLResourceStorageModeShared
        deallocator:nil];
```

On Apple Silicon's unified memory architecture, this avoids any data copy: the GPU reads directly from the `mmap`'d virtual pages. The `nil` deallocator indicates that the mmap owns the underlying memory. If the pointer is not page-aligned (e.g., on non-Apple hardware), the implementation falls back to a copy.

5.4 Loading Procedure

(1) `mmap()` the entire `.tag` file with `MAP_PRIVATE` and `PROT_READ`.
(2) Parse the header and tokenizer from the mapped region (pointer arithmetic, no I/O).

(3) Copy dense f64 arrays (embeddings, norms, biases) to heap-allocated buffers.
(4) Iterate the tensor directory: for each of the 97 tensors, call `metal_register_weights_nocopy()` with pointers into the mapped data section.

The kernel does not touch the tensor data during loading—it is faulted in by the GPU on first access.

5.5 File Size Comparison

Table 1. Model format comparison for Qwen 2.5 0.5B Instruct.

Format	Size	Precision	Loading
`.gguf` (q8)	0.5 GB	int8	standard
`.eml` (v2)	4.8 GB	f64	`fread` + convert
`.tag` (v1)	2.0 GB	f16+bits	`mmap` zero-copy

6 Experimental Results

All experiments use Qwen 2.5 0.5B Instruct (24 layers, $d_{\text{model}} = 896$, $d_{\text{ff}} = 4864$, vocab= 151936) on an Apple M4 Max. The prompt is "What is a tag system?" with 8 generated tokens. Output in all configurations: "A tag system is a type of data".

6.1 Optimization Progression

Table 2. Cumulative effect of optimizations on matmul time and wall clock (8 tokens generated).

Configuration	Matmul	Wall	Speedup
Baseline (naïve C)	42,858 ms	63 s	1.0×
+ fast_exp + ILP + prune	29,774 ms	43 s	1.4×
+ GCD 16 threads	3,611 ms	16 s	11.9×
+ Metal GPU	438 ms	16.5 s	97.8×
+ `.tag` mmap	668 ms	12.9 s	—

The matmul speedup from baseline to GPU is 97.8×. Note that the `.tag` format improves wall-clock time (12.9 s vs. 16.5 s) by eliminating the f64→f16 conversion and GPU upload phase, while inference time remains comparable.

6.2 Time Breakdown by Production Rule

Table 3 shows the time spent in each production rule when using the Metal GPU backend with `.tag` loading.

The four matmul-heavy production rules (GATE_UP, DOWN_PROJ, QKV_MATMUL, OUT_PROJ) account for 85.9% of compute time. Non-matmul operations (norms, residual adds, softmax, RoPE) are negligible at 2.2%.

Table 3. Production rule profile (Metal GPU, `.tag` format, 8 generated tokens, 32 forward passes).

Rule	Count	Time (ms)	%
EMBED	32	0.0	0.0
RMSNORM_ATTN	768	1.1	0.2
QKV_MATMUL	768	111.4	16.7
BIAS_ADD	768	0.4	0.1
ROPE	768	0.2	0.0
KV_STORE	768	0.4	0.1
ATTENTION	768	11.1	1.7
OUT_PROJ	768	103.5	15.5
RESIDUAL_ATTN	768	0.2	0.0
FFN_NORM	768	0.9	0.1
GATE_UP	768	198.6	29.8
SILU_MUL	768	5.5	0.8
DOWN_PROJ	768	159.8	23.9
RESIDUAL_FFN	768	0.2	0.0
FINAL_NORM	32	0.0	0.0
LM_HEAD	32	74.4	11.1
Total		667.5	100.0

Table 4. Loading performance: `.eml` vs. `.tag`.

Format	File	Wall	CPU time
`.eml` (fread + upload)	4.8 GB	16.1 s	13.9 s
`.tag` (mmap zero-copy)	2.0 GB	12.9 s	11.2 s

6.3 Format Loading Comparison

7 Discussion

7.1 Tag System as Control Model

The tag system is the *scheduling layer*: it determines the order of operations. The 315 sequential steps per token are a faithful representation of the transformer's dataflow dependencies. No step can be reordered without violating data dependencies—the tag system encodes the *minimum* sequential structure.

Within each production rule, however, the evaluation is conventional: SIMD instructions, GPU warps, and multi-core parallelism. We call this *tag-system-scheduled, natively-evaluated*: the control model is Post's (1943) 2-tag system, the evaluation model is contemporary hardware.

7.2 Why a Tag System?

Beyond theoretical interest, the tag system provides:

- **Explicit dataflow.** Every operation and its dependencies are visible in the production rules. There are no hidden control paths.
- **Deterministic step count.** Exactly 315 steps per token, invariant of input. This makes performance analysis trivial: profile each production rule independently.

- **Composability.** New operations (e.g., mixture of experts, sliding-window attention) are added by defining new production rules and extending the alphabet.
- **Formal foundation.** The system's computational universality is guaranteed by Cocke and Minsky's theorem [2].

7.3 Limitations

The sign-magnitude representation in f64 is space-inefficient compared to standard quantization formats (q4, q8). The `.tag` format mitigates this with f16 storage, but a 0.5B model still occupies 2.0 GB. The CPU path remains necessary for non-matmul operations, creating a CPU–GPU synchronization boundary at every production rule that involves a matmul.

8 Related Work

Tag systems and universality. Post [1] introduced tag systems as a model of computation. Cocke and Minsky [2] proved 2-tag universality. De Mol [8] surveys the historical development. To our knowledge, no prior work uses tag systems for scheduling neural network computations.

EML operator. Odrzywołek [4] showed that $\mathrm{eml}(x, y) = e^x - \ln y$ can derive all elementary functions. Our sign-magnitude encoding is related: the inner product is computed as $\exp(\ln|a| + \ln|b|) \cdot \mathrm{sign}(a) \cdot \mathrm{sign}(b)$, which factors through exp and ln.

Efficient inference. `llama.cpp` [9] and `ggml` provide highly optimized quantized inference. Our work is not competitive on throughput; it explores a different point in the design space where the control model is an explicit formal system.

Memory-mapped model loading. The `gguf` format supports `mmap`-based loading. `safetensors` [10] also uses memory mapping. Our `.tag` format extends this to zero-copy GPU buffer registration via Metal's `newBufferWithBytesNoCopy`, avoiding even the kernel-to-GPU copy on unified memory architectures.

Classical optimization sources. Our optimizations draw explicitly from: Aho et al. [5] (peephole optimization), Stepanov and McJones [6] (semigroup reduction), and Dechter [7] (constraint propagation for pruning).

9 Conclusion

We have shown that a 2-tag system can serve as the explicit control model for transformer inference. The engine generates correct text from a 0.5B-parameter model using 315 tag steps per token, with each step corresponding to one transformer sub-operation. Five classical optimization techniques—peephole rewriting, semigroup reduction, constraint-propagation pruning, multi-core dispatch, and GPU offloading—bring the matmul kernel from 42.9 s to 0.44 s (97.8×). The `.tag` model format with `mmap` and zero-copy Metal registration reduces file size by 58% and loading time by 20%.

The system demonstrates that even the simplest Turing-complete formalism—a 2-tag system with deletion number $m = 2$ and 18 symbols—can organize the control flow of a modern neural network. Speed is not the primary contribution; rather, the work establishes that formal models of computation from the 1940s remain applicable as scheduling substrates for contemporary AI systems.

References

[1] E. L. Post. Formal reductions of the general combinatorial decision problem. *American Journal of Mathematics*, 65(2):197–215, 1943.

[2] J. Cocke and M. Minsky. Universality of tag systems with P=2. *Journal of the ACM*, 11(1):15–20, 1964.

[3] Qwen Team. Qwen2.5 Technical Report. *arXiv preprint arXiv:2412.15115*, 2024.
[4] A. Odrzywołek. All elementary functions from a single binary operator. arXiv:2603.21852, 2026.
[5] A. V. Aho, M. S. Lam, R. Sethi, and J. D. Ullman. *Compilers: Principles, Techniques, and Tools.* Pearson, 2nd edition, 2006.
[6] A. A. Stepanov and P. McJones. *Elements of Programming*. Addison-Wesley, 2009.
[7] R. Dechter. *Constraint Processing*. Morgan Kaufmann, 2003.
[8] L. De Mol. Tag systems and Collatz-like functions. *Theoretical Computer Science*, 390(1):92–101, 2008.
[9] G. Gerganov et al. llama.cpp: LLM inference in C/C++. https://github.com/ggerganov/llama.cpp, 2023–2026.
[10] Hugging Face. Safetensors: Simple, safe way to store and distribute tensors. https://github.com/huggingface/safetensors, 2023.